Predictor de saltos

Analisis de implementaciones, riesgos y beneficios

Javier García
Estudiante de Ingeniería Informática
Universidad Rovira I Virgili
Tarragona, España
javigd.dtk@gmail.com

Christian Martínez
Estudiante de Ingeniería Informática
Universidad Rovira I Virgili
Tarragona, España
Christian.mr7@gmail.com

Abstracto—

El tratamiento de saltos en las etapas del pipeline genera una gran pérdida de tiempo. Hasta que no se ejecuta la instrucción no se puede determinar si se procede con el salto o se continúa con la ejecución secuencial. Para evitar este estancamiento se implementan los predictores de saltos. Se intenta predecir en la etapa de fetch la decisión final que se tomará en la etapa de ejecución. La predicción puede basarse en un comportamiento predefinido o en la historia de los resultados de salto anteriores. De este modo se evita la pérdida de tiempo, cargando las instrucciones que se prevé se deberán ejecutar una vez evaluado el salto. No obstante, la predicción de saltos puede ser contraproducente si no se tiene un alto índice de aciertos. Cada predicción falsa implicará una penalización dura. Se tendrá que deshacer el error y después se deberá continuar con la ejecución desde donde realmente correspondía. Por consiguiente se tratará de conseguir el mayor índice de aciertos mediante distintas técnicas de predicción que trataremos en este artículo.

1. INTRODUCCION

Mejorar el pipeline es una de las maneras más efectivas de incrementar el rendimiento del procesador. Muchas son las mejoras que se han ido introduciendo, reduciendo el tiempo de ejecución drásticamente. El incremento en las vías de las diferentes etapas y la aparición de la ejecución fuera de orden son dos claros ejemplos. Sin embargo, contra más grande y complejo se hace el pipeline, la repercusion negativa de las intrucciones de salto se intensifica. Esto es consecuencia del trato que reciben estas instrucciones. Cuando un branch es detectado, el pipeline se detiene esperando a que la instrucción de salto se ejecute y se determine si tomar el salto o continuar con la ejecución secuencial. Así mismo, para saltos no condicionales también se debe esperar a que se calcule la dirección de la siguiente instrucción.

Dos son los objetivos principales para resolver el problema: en primer lugar, determinar el resultado de la operación de salto antes de que esta se ejecute, para empezar a cargar las instrucciones pertinentes con antelación.

El segundo objetivo será resolver la dirección a la que hay que saltar. Cuando un salto es tomado, se necesita conocer la dirección de destino, que se obtiene al ejecutar la instrucción. Esto conlleva una parada del pipeline, hasta que se obtiene dicha dirección. Para resolver esta problemática será necesario un mecanismo que proporcione la dirección tan pronto como sea posible. El branch target prediction se encarga de facilitar la información, más adelante se explicará su funcionamiento.

La predicción de saltos determina la manera en que se intentara solventar el estancamiento del pipeline.

Existen dos tipos de predicción: la estática y la dinámica. La predicción estática se implementa en tiempo de compilación. Se denomina estática, porque se conoce la acción a realizar para cada salto antes de que el programa se ejecute. Además, la decisión es inalterable en tiempo de ejecución. Estas características hacen que su implementación sea sencilla, de bajo coste y que la toma de decisiones en el pipeline se lleve a cabo con mucha rapidez. Un ejemplo de estos predictores es el Always Taken, que decide tomar todos los saltos independientemente de su naturaleza. Este algoritmo alcanza una media de acierto del 60%, frente el 40% proporcionado por la implementación contraria (Never Taken). Muchas son las variantes de los predictores estáticos, pero el problema de éstos es su bajo porcentaje de acierto. La precisión máxima en el mejor de los casos es del 75%. En cambio, con la predicción dinámica se consiguen porcentajes superiores al 80%.

Un predictor dinámico aprende de los saltos realizados anteriormente en tiempo de ejecución. Puede detectar patrones de comportamiento de un bloque de programa, o de un salto en concreto. Para ello se requerirá el uso de un hardware más complejo que en el caso del estático, lo que conlleva un coste computacional y de hardware más elevado, a cambio de una mayor fiabilidad. En general se prefiere asumir los costes adicionales que supone la implementación dinámica, pues el ratio de acierto es de entre el 80% y el 99% dependiendo del predictor. Teniendo en cuenta que el rendimiento del procesador aumenta exponencialmente cuando el índice de aciertos crece, es normal que se prefiera este tipo de predicción.

La sección dos proporciona una visión más concreta de la repercusión que tienen las instrucciones de saltos. La sección explica la predicción estática y los principales predictores que la implementan. En la sección cuatro se describe la predicción dinamica y las principales estructuras involucradas. La sección

cinco analiza los principales predictores dinámicos. La sección seis detalla las características de predicción de saltos de varios procesadores comerciales. En la sección siete se muestran las conclusiones del análisis realizado sobre la predicción de saltos.

2. MOTIVACIÓN

Cuando un programa es ejecutado, las instrucciones de las que se compone se cargan en el pipeline de forma secuencial. Se carga la instrucción a la que apunta el PC y después se aumenta el valor de PC señalando a la instrucción contigua. Este proceso se realiza sucesivamente hasta que el buffer de fetch esté lleno.

El pipeline compuesto por diferentes etapas, va tratando las instrucciones, haciéndolas pasar de etapa en etapa a medida que estas han sido tratadas según la finalidad cada una. De esta manera se cargan en fetch para luego ser decodificadas, despachadas y ejecutadas. El problema aparece cuando el programa contiene una instrucción de salto condicional, pues ésta permite que la ejecución no sea secuencial. Cuando esto ocurre, se crea una bifurcación en la que se debe elegir si se continúa con la ejecución secuencial o se toma un salto a un bloque de instrucciones no contiguo. No se sabe cuál de las dos opciones tomar hasta que el salto no se ejecuta y se obtiene la respuesta. Por lo tanto el pipeline se detiene hasta saber dónde se encuentra la siguiente instrucción a tratar. Ello conlleva a una pérdida de tiempo abismal teniendo en cuenta que en un programa una de cada diez instrucciones es de salto.

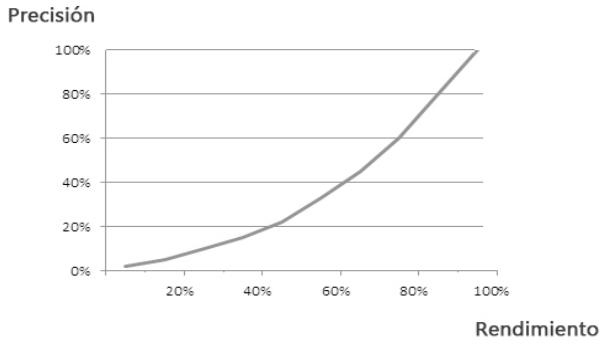
Este problema es característico de los saltos condicionales. No obstante, aun sabiendo que hay que tomar el salto, no se sabe a qué lugar hay que saltar hasta que se ejecute y se calcule la dirección. Como consecuencia se vuelve a producir estancamiento hasta que se consigue la dirección de salto. Esto afectará tanto a saltos condicionales como no condicionales, conllevando un retardo de dos ciclos normalmente.

La predicción de saltos evita estos estancamientos adivinando si los saltos condicionales deben ser tomados antes de que se ejecuten, para que el flujo de instrucciones no se detenga. En paralelo la predicción de destino de salto se encarga de proporcionar la dirección a la que se debe saltar, resolviendo el tiempo de espera en saltos condicionales y no condicionales.

Si bien la predicción disminuye el tiempo de ejecución del programa rotundamente, es necesario que sea correcta. Cada fallo en la predicción implicará el vaciado de todo el pipeline para la posterior carga de las instrucciones correctas. Este problema se acentúa a medida que la tecnología avanza y se dota al pipeline de más capacidad. Cuando se amplía grado de superescalar o el tamaño de las colas de instrucciones, una mala predicción penaliza el rendimiento del procesador en mayor medida. Entonces la unidad de mesura que nos indica la fiabilidad de un predictor de saltos será el porcentaje de aciertos que nos proporcione. Cada punto porcentual a la alza implicará un crecimiento exponencial del rendimiento del procesador.

Estas propiedades hacen de la predicción de saltos un punto fundamental de investigación en la mejora de la eficiencia.

La siguiente gráfica muestra la relación orientativa entre aciertos y rendimiento.



3. PREDICCIÓN ESTÁTICA

La predicción se lleva a cabo en tiempo de compilación. El compilador mediante un algoritmo decide que saltos serán tomados y cuáles no. Por ejemplo, un algoritmo puede consistir en determinar que para cualquier salto la decisión será tomarlo (Always Taken). Se denomina estática porque la predicción se conoce antes de que el programa se ejecute y es inalterable.

Este tipo de predictores requieren un hardware mínimo: El necesario para el vaciado del pipeline y una modificación del ISA. La modificación del ISA permite guardar un campo extra en la instrucción para indicar la decisión sobre el salto.

Aunque ganan en simplicidad y coste (tanto monetario como computacional), los mejores resultados obtenidos aportan una precisión del 75% y por este motivo no suelen utilizarse.

A continuación se describen los tres predictores estáticos más utilizados. Pese a que existen multitud de implementaciones, con ellos será suficiente para entender el concepto.

3.1 NEVER TAKEN

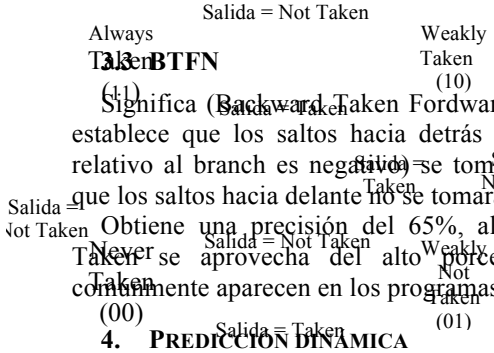
Como su nombre indica este predictor ignora todos los saltos condicionales por defecto, por tanto, cuando una instrucción de este tipo llega al pipeline, siempre se continuará la carga con las instrucciones contiguas siguientes al PC.

Es el peor en cuanto a precisión de los que aquí se analizan, obteniendo una media del 40% de aciertos. Un resultado muy pobre que conlleva el vaciado del pipeline el otro 60% de los branch.

3.2 ALWAYS TAKEN

Esta variante es la implementación contraria al Never Taken, lo que implica que la decisión será siempre la de tomar el salto. En este caso se continuará con la ejecución especulativa de las instrucciones a partir de la dirección que indica el branch.

La media de aciertos de este predictor se sitúa en torno al 60%, debido a que la mayor parte de saltos condicionales que contiene cualquier programa forman parte de un bucle, en el que se deberá tomar el salto hacia atrás repetidas veces antes de salir de él.



3.3 BTFN

Significa (Backward Taken Fordware Not). Este algoritmo establece que los saltos hacia detrás (aquellos donde offset relativo al branch es negativo) se tomarán siempre, mientras que los saltos hacia delante no se tomarán en ningún caso.

Obtiene una precisión del 65%, al igual que el Always Taken se aprovecha del alto porcentaje de bucles que comúnmente aparecen en los programas.

4. PREDICCIÓN DINÁMICA

La predicción dinámica se basa en el aprendizaje en tiempo de ejecución. En la decisión de tomar un salto, utiliza como referencia el comportamiento histórico de estas instrucciones. Para conseguirlo es necesario dotar al procesador de un hardware más complejo, que se encargue de recopilar la información y modificarla. Para que sea funcional, la decisión debe tomarse lo antes posible. Por ello, el proceso se lleva a cabo en la etapa de fetch. Con este mecanismo se consigue aumentar la precisión de las predicciones superando el 80% de aciertos.

El diseño de predictores dinámicos está sujeto al poco espacio del que se dispone para la integración del circuito y se debe valorar si los beneficios de añadir más hardware compensarán el coste y los inconvenientes ocasionados.

Normalmente se utiliza la dirección de la instrucción de salto para identificarla y registrar su comportamiento. De todas formas, hay predictores dinámicos que obvian esta información debido a que se basan en la historia global. Éstos observan en el comportamiento conjunto de todos los saltos para detectar un patrón, sin identificar instrucciones. Así encontraremos predictores dinámicos que utilizan la información local (individualizan comportamiento por instrucción), global, o más complejos que basarán sus predicciones en ámbito local y global simultáneamente. Con el fin de registrar esta información se utilizan diferentes estructuras de hardware. Cada una de ellas tiene un propósito, pudiéndolas combinar para alterar o coordinar su función.

Ahora pasaremos a definir las estructuras de más relevancia y que utilizan los predictores dinámicos del apartado cinco.

4.1 BTB (BRANCH TARGET BUFFER)

Como se ha comentado, existe un estancamiento cuando se debe obtener la dirección de salto de un branch. Esta tabla se utiliza para para almacenar las direcciones destino de estas instrucciones. La primera vez que una instrucción de branch se ejecuta, la dirección de destino a la que se ha saltado se guarda en la BTB. Para poder acceder a ella la siguiente vez que se cargue el mismo branch, ésta se indexa a partir del PC de la instrucción de salto. Ésta es la forma más sencilla de branch target prediction. En ella se produce aliasing y da lugar a predicciones equivocadas. Para evitarlo, en la actualidad se usan varias caches asociativas con múltiples vías para su implementación. Normalmente se utilizan varios BTB dependiendo del tipo de salto que se quiera almacenar (ej. Saltos condicionales, llamadas a ruinas, returns etc.) .

Ej.

BTB	
Hash(@branch)	Branch target

ONE-BIT COUNTER

Tan sencillo como un bit que almacena el estado del último salto ejecutado. A partir de este se emprende la ejecución especulativa confiando en su valor. En la implementación básica se almacena en la cache de instrucciones.

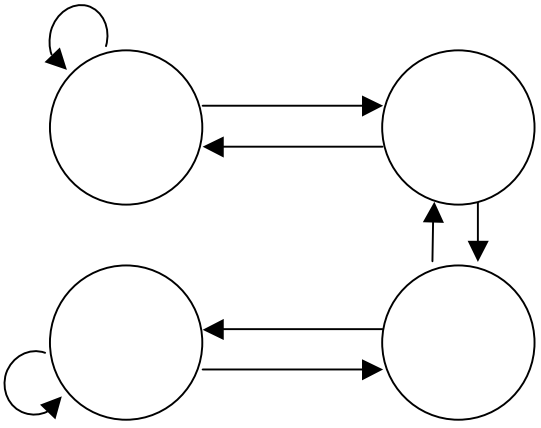
Proporciona una precisión aceptable en comportamientos repetitivos como bucles, aunque siempre fallará la entrada y la salida. Como contrapartida es nefasto en situaciones de histéresis, fallando la predicción a cada salto (ej, 0101010).

Las diferentes estructuras aquí analizadas, utilizan este contador, o su versión mejorada (two-bit counter) para tomar las decisiones de salto.

4.2 TWO-BIT SATURATING COUNTER

Es una versión ampliada del one-bit counter que almacena dos bits en lugar de uno. Se encarga de almacenar el comportamiento del salto utilizando el bit de mayor peso. Los valores de estos bits reflejan con mayor precisión la frecuencia de veces que se ha saltado. Así tenemos que el valor 11 indica Always Taken, 10 Weakly Taken, 01 Weakly Not Taken y 00 Never Taken. De esta manera cuando una instrucción de salto se ejecuta, se incrementa el valor si se ha tomado el salto, y se decrementa de lo contrario. Ahora será necesario que se produzcan dos falsas predicciones para cambiar la decisión de cuando éste tiene el mismo comportamiento en la mayoría de casos. Nótese que ahora en un mismo bucle se acertarán todas las entradas al bucle fallando únicamente en la salida. Aun así, como el one-bit counter, no responde bien a las situaciones en las que el salto alterna la decisión cada vez. En el mejor de los casos fallará la mitad de veces, y en el peor todas. Ésto depende del estado en que se encuentre cuando empieza éste tipo de comportamiento.

Diseño esquemático del comportamiento del two-bit counter:



4.3 BHT (BRANCH HISTORY TABLE)

Ubicada en la etapa de fetch consiste en una tabla indexada a partir de un hash aplicado al PC del salto. Contiene la información que determina la acción a realizar cuando llega el salto al buffer de fetch. En su versión más simplificada, además del índice, almacena un bit que proporciona la última acción realizada por el branch. También se puede utilizar un 2-bit counter, o implementaciones más complejas con ánimo de mejorar su fiabilidad.

Ej.

BHT	
Hash(@branch)	1-bit counter

4.4 BHR (BRANCH HISTORY REGISTER)

Estructura diseñada para almacenar patrones de comportamiento. Se indexa a partir del PC de la instrucción de salto. En ella se define un tamaño n y a medida que se ejecutan los saltos se añade el resultado (tomado o no) substituyendo al bit más antiguo. De esta manera que se puede registrar una historia del comportamiento de los últimos *n* saltos detectando patrones. El BHR se utiliza en combinación con otras estructuras como la PHT (Pattern History Table) para asociar el punto actual en que se encuentra del patrón y la decisión a tomar (Ej. Two-Level adaptative).

Ej.

BHR	
Hash(@branch)	Pattern

4.5 PHT (PATTERN HISTORY TABLE)

Se constituye por un two-bit saturating counter en cada entrada de la tabla, indexado por un patrón de comportamiento. Normalmente asociada a un BHR permite determinar qué acción tomar en un determinado punto de un patrón.

Ej.

PHT	
Hash(pattern)	2-bit counter

5. PREDICTORES DINÁMICOS

Se han escogido diferentes tipos de predictores dinámicos los cuales tienen diferencias significativas entre sí. El objetivo es analizar las repercusiones de los distintos usos aplicados a las mismas estructuras.

5.1 ONE-BIT COUNTER PREDICTORS

Utiliza un bit para registrar el comportamiento de los saltos. Cuando un branch entra en el pipeline, se lee el valor del bit y se empieza con la ejecución especulativa considerando su valor. Una vez se ejecuta la instrucción de salto se actualiza el contador indicando si se ha tomado el salto.

Se puede realizar una implementación local o global. Para la primera se utiliza una BHT con un one-bit counter permitiendo discriminar entre branch . Para el segundo caso, la manera más fácil es guardar el valor en la cache de instrucciones, aunque da lugar a una predicción aleatoria.

Con este procedimiento se consigue acertar la mayoría de saltos en un loop, pero siempre se falla en la entrada y la salida.

5.2 BI-MOD

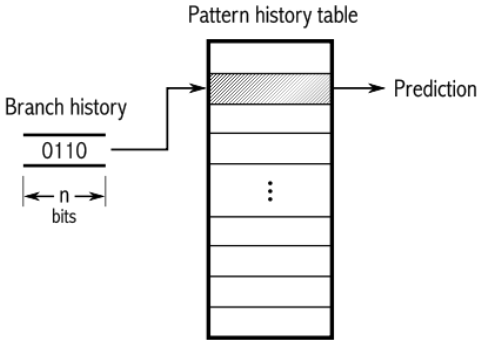
Utiliza una tabla de two-bit counters para almacenar el comportamiento de cada branch. Para identificar la entrada en la tabla que corresponde, aplica un hash a la dirección del PC. Al realizar el hash se puede producir aliasing, dando lugar a una predicción aleatoria. No obstante, obtiene mejores resultados que los predictores con un solo bit de historia.

5.3 TWO-LEVEL PREDICTORS

Los predictores anteriores se centraban en obtener la máxima información de un salto en particular. Ahora es necesario profundizar, intentar desentrañar el entorno del salto. Obteniendo la relación entre saltos se pueden detectar patrones de comportamiento y obtener predicción más precisa.

En los predictores two-level se utiliza un esquema con dos niveles de predicción. Este esquema se compone de uno o más BHR donde el patrón registrado se utiliza para indexar el comportamiento del salto en una PHT. Por consiguiente, obsérvese que en la PHT se almacenarán los comportamientos del salto dependiendo en el punto del patrón en el que se encuentre.

Ej.



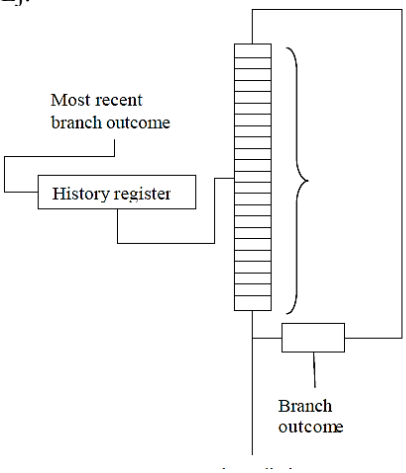
A continuación se detallan diferentes implementaciones de basadas en el esquema two-level:

- **GA Predictor**

  Utiliza un BHR para guardar la historia global de las instrucciones de branch. En este registro se guardan las salidas de todos los saltos sin identificarlos individualmente. Por tanto, establece una correlación entre el salto actual y los demás. Es muy útil cuando el resultado de los saltos es repetitivo. Aun así, se da lugar a colisiones cuando

dos bloques de código tienen el mismo patrón. Otra desventaja es que si los diferentes saltos se comportan de forma irregular, la predicción no será tan buena.

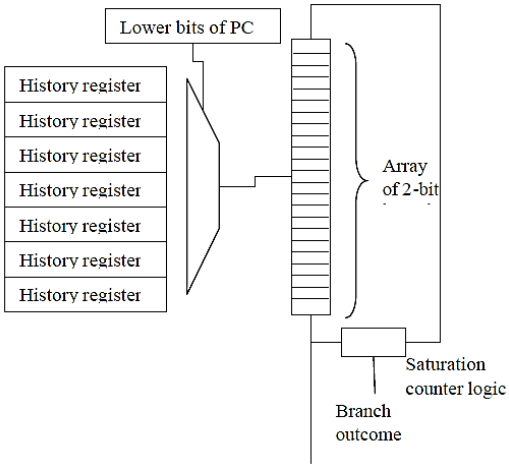
Ej.



- **PA Predictor**

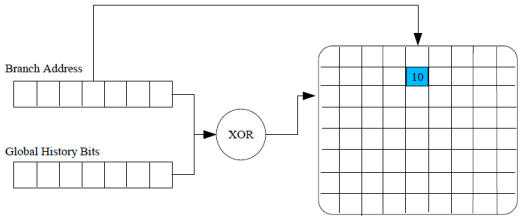
  A diferencia del GA, éste es de ámbito local, identifica los saltos a través de su dirección en el PC. Este predictor funciona bien para detectar patrones de comportamiento individual, por ejemplo, si un salto es tomado con una secuencia 110011001100, será detectada fácilmente. Por contra, se puede producir aliasing entre saltos, ocasionando predicciones aleatorias.

Ej.



- **GShare Predictor**

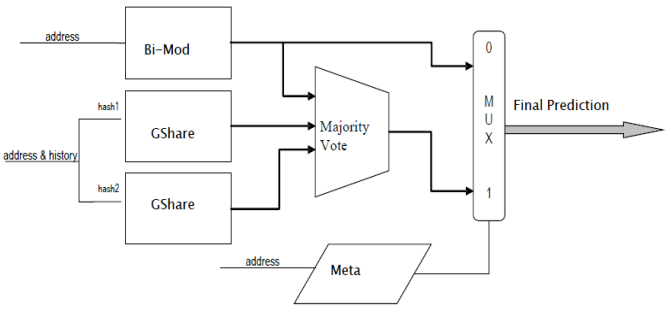
  El GShare es una forma más avanzada de un two-level predictor. Utiliza la historia global almacenada en un BHR y hace una XOR con una parte de la dirección del PC de la instrucción. El hecho de hacer la XOR mejora el GA reduciendo la posibilidad de colisión entre patrones de bloques de código diferentes. Por contra requiere la utilización de más hardware.



5.4 HÍBRIDOS

Los predictores híbridos son de los que tienen un diseño de mayor complejidad. Se intenta obtener un resultado óptimo mediante la combinación de predictores más simples. Sin embargo, la viabilidad de éstos se pone en duda. Hay que valorar el coste computacional y de hardware que conlleva, y si la diferencia compensa lo que un predictor más rápido y sencillo consigue.

En la siguiente figura se muestra un ejemplo de predictor hibrido, el 2bc-gskew. Finalmente encontramos un predictor que consigue eliminar el aliasing. Para ello utiliza dos GShare a los cuales asigna una función de hash diferente y un bi-mod. Se envían las tres predicciones a un módulo que elegirá la opción que gane en mayoria de votos. Si los GShare disciernen, significa que se ha producido una colisión en una de las tablas. En este punto se utiliza el bi-mod como voto decisivo. Así mismo, el Meta junto al bi-mod se utilizan para solventar el problema de aprendizaje de los GShare. Un GShare, necesita *n* instrucciones de salto de aprendizaje y se deben tomar decisiones durante ese tiempo.



6. PROCESADORES COMERCIALES

Con el tiempo, los procesadores han ido introduciendo mejoras y nuevas combinaciones en los predictores de saltos que implementan. A continuación se detallan las propiedades de varios procesadores comerciales. Para cada uno se identificarán el predictor de saltos que utilizan, el predictor de destino de salto y la penalización en ciclos a cada fallo en una predicción.

- **Intel P2**

  BTB: 512 entradas (16 vías * 32 entradas).

  Predicción de saltos: Un two-level adaptative con 4 bits de historia. No implementa patrón de

reconocimiento para saltos indirectos ni llamadas indirectas.
Penalización: entre 10 y 20 ciclos.

- **PM**
  BTB: Utiliza 2048 entradas (4 vías * 512 entradas) para saltos no condicionales y branch sin comportamiento de loop. Para saltos con comportamiento de loop posee 128 entradas (2 vías * 64 entradas). Para saltos indirectos y llamadas 8192 entradas (4 vías * 2048 entradas). Finamente los returns los guarda en una pila de 16 entradas.
  Predicción de saltos: Para saltos condicionales, consistente en un predictor hibrido formado por un predictor de dos niveles y un contador de loops. Una instrucción de salto se determina como loop cuando va n-1 veces en la misma dirección. Posee además un reconocimiento de patrones para saltos indirectos y llamadas indirectas (pero no para saltos).
  Penalización: por fallo en predicción de 13 ciclos.

- **Intel Nehalem**
  BTB: Tiene dos niveles, como un sistema de dos niveles de cache. Hay diferentes buffers para los diferentes tipos de saltos.
  Predicción de saltos: Patrón de reconocimiento de saltos condicionales consta de un predictor hibrido formado por uno o mas two-level predictors y un loop counter. Además contiene un mecanismo para saltos indirectos y llamadas indirectas.
  Penalización: 17 ciclos.

- **Amd Bulldozer**
  BTB: Consta de dos niveles. El level-1 es una cache asociativa de 512 entradas (4 vías * 128 entradas). El level-2 proporciona 5120 entradas (4 vías * 1024 entradas)
  Predicción de saltos: Mecanismo hibrido que combina la historia local y la global.
  Penalización: 20 ciclos para saltos condicionales e indirectos. Cuando los saltos son incondicionales o returns la penalización es de 15 ciclos.

7. CONCLUSIONES

El predictor de salto se ha convertido en uno de los componentes más destacables del procesador. En los procesadores modernos, una buena predicción tiene una repercusión directa muy significativa sobre el rendimiento de éstos. Por otro lado, una mala predicción conlleva un deterioro notorio del rendimiento.

Por tanto, la característica líder en un predictor es la precisión o índice de acierto. Cada modificación en el diseño y estructura de un predictor se ha hecho priorizando ésta propiedad. Al mismo tiempo que avanzaba la tecnología en éste campo, salían a la luz los problemas más relevantes. En primer lugar el coste de hardware, directamente relacionado con la complejidad y dinamismo del predictor y a su vez inversamente relacionado con la celeridad de las predicciones.

Así pues, los primeros predictores implementados eran de carácter estático, y por lo tanto gozaban de una estructura simple en hardware. Esto les permitía recibir el outcome de una manera rápida, pero aun así la precisión se veía seriamente penalizada. Teniendo en cuenta las limitadas posibilidades que nos brindaban estos predictores, únicamente se pudo optimizar en base a la frecuencia de aparición de cada tipo de salto dentro del código.

En este punto apareció la posibilidad de cambiar el comportamiento del predictor durante el flujo de instrucciones por el pipeline. Los dos problemas principales que plantearon los predictores dinámicos fueron la correlación y el aliasing. La decisión de llevar a cabo una predicción global o local respecto cada dirección planteaba un sistema individualizado o por el contrario correlativo a otros saltos. La duda se solucionó dando lugar a predictores basados en ambas tecnologías. No obstante, el problema del solapamiento de referencias persistía. El desarrollo de predictores híbridos e inteligentes en los últimos años ha permitido exprimir al máximo la eficiencia de éstos y a su vez reducir a niveles insignificantes el aliasing. Por contrapartida, el coste adicional de fabricación y computacional, pueden llegar a poner en duda su adopción.

Para concluir, destacar la importancia del continuo desarrollo de esta tecnología. Los métodos más avanzados (perceptrones), utilizan esquemas de predicción basados en la simulación de redes neuronales. Éste nivel de complejidad dice mucho sobre la implicación de un buen predictor de saltos en el rendimiento de un procesador actual.

BIBLIOGRAFIA

[1] http://moodle.urv.net/moodle/file.php/37836/Documentos/T23_BranchPred/survey_branchpred_2.pdf
[2] http://en.wikipedia.org/wiki/Branch_predictor
[3] http://en.wikipedia.org/wiki/Branch_target_predictor
[4] http://moodle.urv.net/moodle/file.php/37836/Documentos/T23_BranchPred/intel_arm_architecture.pdf
[5] http://moodle.urv.net/moodle/file.php/37836/Documentos/T23_BranchPred/survey_branchpred_1.pdf
[6] http://www.cs.ucr.edu/~gupta/teaching/203A-09/My6.pdf
[7] http://www.cs.iastate.edu/~prabhu/Tutorial/PIPELINE/branchPred.html
[8] http://users.cis.fiu.edu/~downeyt/cop3402/prediction.html
[9] http://www.inf.pucrs.br/~calazans/graduate/SDAC/saltos.pdf
[10] http://web.cecs.pdx.edu/~alaa/ece587/notes/bpred-6up.pdf